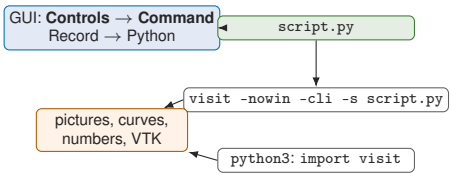
VISIT FROM THE COMMAND LINE AND PYTHON

The CLI and its scripts, recording in the GUI, the Python interface piece by piece, errors and return values, Python 2 scripts in Python 3, NumPy inside VisIt, and VisIt inside your own Python



VisIt 3.5.0 (its CLI is Python 3.13 with NumPy), measured on this series' runs; the 2018 decks' Python 2 scripts ported

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part V • Chapter 32

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

- Run VisIt's Python: interactively, from a script file with arguments, inside a running CLI, and from your own Python.
- Record what you do in the GUI as Python, and turn the recording into a script that loops over time.
- Find any function or attribute without a manual (`help`, `apropos`, `dir`, printing an attributes object).
- Check every step of a script (return values, `GetLastError()`), and know which failures VisIt does not report.
- Port a Python 2 VisIt script of the old decks to VisIt 3.5's Python 3.
- Combine VisIt with NumPy, inside VisIt's CLI or in your own Python.

> Every new picture of a plot and every number of Part V was made by a script (the window screenshots are the old decks'). The GUI is where you design a picture; a script is how you make it again, for every frame and every run.

WHERE THIS CHAPTER COMES FROM

- "8 – Visit from command line and Python" (2018): "Goal: Use visit for big data analysis without downloading the data or using the GUI", on the Killdevil cluster: `module add visit/2.12.3`, `visit -cli -nowin` and `Source("heart_script.py")`, or `visit -cli -nowin -s`; building VisIt with `build_visit --mesa`; recording; a recorded script of IB/11's racetrack with a loop over time; and C. Strickland's "Option 2: Run visit from python": `import visit` into your own Python.
- "6 – Resampling AMR grids to Uniform Grid, exporting to .vtk, and recording in Python" (2018), slides 2–4 and 15–16: "record your python commands in visit 2.5.2"; "Before you start doing anything, click on the execute button"; "Then the code can be modified to loop through time."
- "3B – Analyzing your IBFE simulation" (2018), slides 3–8: record, edit the paths, add a loop, run on the cluster.
- G. Hobson (2020; chapter 31): **Controls** → **Command**, **Record**, **Stop**, copy the code into "a whitespace-sensitive text editor".
- The decks point to Shannon Jones's video how-tos on recording (listed in chapter 0.1, not used here) and to a visitusers.org scripting tutorial (the site is gone).

New here: every claim run in VisIt 3.5.0 without a window, except those about the GUI (the Command window and its recorder: read from VisIt's source and GUI library and the 2018 recordings): the CLI's Python and packages, what functions return when they fail, what the recorder writes, the arguments a script receives, the external `import visit` route; and the scripts of chapters 27, 29–31 and 33 as worked examples.

WHAT CHANGED SINCE THE 2018 DECKS

2018	VisIt 3.5.0
“VisIt’s Python interpreter is … a basic Python 2.x environment … NumPy and matplotlib in particular are absent”	the CLI is Python 3.13.9 with NumPy 2.3.4 and `visit_utils`; matplotlib, SciPy and h5py are still absent (slide 19)
Python 2’s `print "..."`, integer `/`, `xrange` (the decks’ own scripts use none of them)	Python 3: `print(...)`, `//` for integers, `range` (slide 18)
“click on the execute button” to record	the button is **Record** (**Execute** runs the text of its tab; slide 8)
Killdevil’s `visit` module, VisIt 2.5.2 (“record your python commands in visit 2.5.2”), and `visit/2.12.3`	Puma’s `visit/3.4.2` module or your own install (chapter 31); record in the version you will run
the Windows hotfix to `frontend.py` for `import visit`	not checked on Windows here; on Linux the route works with two environment variables (slide 20)
`Source("script.py")` from the CLI prompt	still works; `-s script.py -- arguments` is easier to repeat
visitusers.org’s tutorial	gone; the VisIt manual’s Python chapter, and `help()` and `apropos()` in the CLI

The 2018 decks of the previous slide; VisIt 3.5.0: `wsl/final/ch32_*.sh`.

SIX WAYS TO RUN VISIT'S PYTHON

```
visit -cli                                    # 1. a Python prompt with a VisIt window
visit -cli -nowin                             #    the same without a window: >>> OpenDatabase(...)
visit -nowin -cli -s script.py -- a b k=v     # 2. a script with arguments (the way of this series)
>>> Source("script.py")                       # 3. a script run inside a CLI that is already open
printf 'print(Version())\nexit()\n' | visit -cli -nowin      # 4. commands piped in: ">>> 3.5.0"
visit -movie -scriptfile movie.py ...         # 5. a script inside the movie maker (chapter 30)
python3 my_analysis.py                        # 6. your own Python, with "import visit" (slide on
  import visit)
```

- In the GUI, **Controls** → **Command** gives a seventh: Python typed or recorded in a window, run with **Execute** (slide 8).
- Every CLI run leaves `visitlog.py` in the working folder: the commands it sent, a log of the session.
- A script run with `-s` should end in `sys.exit()`: without it the CLI goes on to its prompt and, started from a terminal, waits there. In a batch job (chapter 31) Slurm gives the script `/dev/null` as input, and with that input the CLI ended at its prompt at once. The exit status was 250 every time, after `sys.exit(0)` too: count what a script wrote instead.

VisIt 3.5.0 `visit -help`; `wsl/final/ch32_cli_helpers.sh`; the prompt and the exit status: `scratch/ch32/verify_agent/v5_runs.sh`, `v9_exitcodes.sh`; Slurm's `sbatch` manual (`--input`).

A SCRIPT'S ARGUMENTS

```
import sys
print(sys.argv)
# visit -nowin -cli -s args.py -- a b k=v
#   ['args.py', '--', 'a', 'b', 'k=v']
# visit -nowin -cli -np 2 -s args.py -- a b k=v
#   ['args.py', '--', 'a', 'b', 'k=v', '-engineargs', '-l;mpirun;-np;2', '-launchengine', '
  localhost']
```

- Everything after `--` is the script's; VisIt keeps the `--` itself in `sys.argv`.
- With a parallel engine VisIt appends its own options: stop reading at `-engineargs` (the scripts of `examples/analysis` do).
- The pattern of this series: positional arguments for the data and the output folder, `key=value` for options, the defaults and their meaning in the script's docstring, printed when the arguments are missing (next slide).

`wsl/final/ch31_parallel.sh` (the CLI's command line with `-np`); `print(sys.argv)` run with and without `-np 2` (VisIt 3.5.0; `scratch/ch31/verify_agent/v3_cli.sh`, `scratch/ch32/verify_agent/v5_runs.sh`).

READING THE ARGUMENTS

The argument lines of `examples/analysis/visit_queries.py`; every VisIt script there has the same loop (`visit_3d.py` and `resample_export.py` take three positional arguments):

```
args = []
for a in sys.argv[1:]:
    if a in ("-engineargs", "-launchengine"):
        break
    if a != "--":
        args.append(a)
if len(args) < 2:
    print(__doc__); sys.exit(1)
DB, OUT = args[0], args[1]
opt = dict(a.split("=", 1) for a in args[2:])
```

Then `opt.get("lim", "auto")` (a key of `visit_movie_frames.py`) and the like give each option with its default; `__doc__` is the docstring at the top of the file, so the help a user sees is the one written there.

RECORDING IN THE GUI

Controls → **Command . . .** opens the Command window, laid out in VisIt 3.5 as in 2018. Its **Commands** box has **Record**, **Pause** and **Stop**, **Store commands in** (**Active tab**, **First empty tab** or **Macros**) and **Append commands to existing text**; below it are eight numbered tabs, each a text box with **Execute**, **Clear** and **Make macro**, and a **Macros** tab with **Update macros** and **Clear**.

1. **Record**, then make the picture in the GUI as usual; **Stop**. The Python for every step appears in the active tab's box.
2. Copy it into an editor ("a whitespace-sensitive text editor", 2020; the 2018 deck: "Cut and paste python to a text editor (but not the software text edit)").
3. Keep what you need, put the paths in variables, add a loop over time (slide 17), save a picture per step.
4. **Execute** runs whatever is in its tab's box, recorded or typed: a quick way to test a few lines against the open session.

> The 2018 decks said "Before you start doing anything, click on the execute button". The button that records is **Record**; **Execute** runs the text. "You may want to run through what you are going to do before recording, however, and then actually record when you know you can go through the procedure without mistakes" still holds.

Button and tab names from VisIt 3.5.0's GUI library (`wsl/final/ch32_cmdwin_strings.sh`), their layout from VisIt's `QvisCommandWindow.C` and the 2018 deck's screenshot (VisIt 2.13.0, slide 11); the 2018 decks; G. Hobson (2020), slides 6–7.

WHAT THE RECORDER WRITES, AND WHAT TO KEEP

```
# recorded (2018, IB/11's racetrack; the paths shortened)
OpenDatabase("localhost:/.../4-VP_heart_viz_IB2d/lag_data.visit",
   0)
OpenDatabase("localhost:/.../4-VP_heart_viz_IB2d/dumps.visit", 0)
AddPlot("Pseudocolor", "Omega", 1, 0)
DrawPlots()
ActivateDatabase("localhost:/.../4-VP_heart_viz_IB2d/lag_data.
  visit")
AddPlot("Mesh", "heart_peri_512_vertices", 1, 0)
AddPlot("Mesh", "heart_race_512_vertices", 1, 0)
AddPlot("Mesh", "heart_tube_512_vertices", 1, 0)
AddPlot("Mesh", "markers_512_vertices", 1, 0)
AddPlot("Mesh", "vp_aforce_512_vertices", 1, 0)
DrawPlots()
SaveWindowAtts = SaveWindowAttributes()
SaveWindowAtts.outputToCurrentDirectory = 0
SaveWindowAtts.outputDirectory = "./visit_images"
...                      # 15 more fields
SetSaveWindowAttributes(SaveWindowAtts)
SaveWindow()
```

- `localhost:` and the absolute path of the machine it was recorded on: change the path (chapter 31); `localhost:` is the machine VisIt runs on and can stay (a `localhost:` path opened in 3.5.0).
- `AddPlot(type, variable, 1, 0)`: the two flags are `inheritSIL` and `applyOperators` (whether the new plot takes the active plot's SIL restriction, and its operators); `AddPlot(type, variable)` is enough.
- The recorder sets every field of an attributes object, including the defaults (the 2018 loop set 17 fields of `SaveWindowAttributes`). Keep the ones you changed; the rest only hides them.
- The zoom is recorded, and one of its lines must go: next slide.
- The Subset plot of `levels` that opening `dumps.visit` adds is not in the recording but is in the session: delete it in the script (chapter 30).

"8 – Visit from command line and Python" (2018), slides 12–13; VisIt 3.5.0 `help(AddPlot)`; the `localhost:` path: `scratch/ch31/verify_agent/v3_cli.sh`.

THE ZOOM IN A RECORDING

- The zoom is recorded. The zoom icon becomes `SetWindowMode("zoom")`; in a CLI without a window that line kills the viewer (“VisIt’s viewer has terminated abnormally!”), and every later call fails (“VisIt’s viewer is not running!”).
- The view the zoom gave is written as `View2DAtts` fields and `SetView2D(View2DAtts)` between `# Begin spontaneous state` and `# End spontaneous state`. Run without a window, such a block sets the view as it should.
- The 2020 deck: “commands to do with spontaneous states … do not work on the cluster”. Delete the `SetWindowMode` lines; keep the view, or set it with numbers (`windowCoords`, chapter 30).

G. Hobson (2020), slide 13; VisIt’s recorder (`Logging.C`: the spontaneous-state blocks, `SetWindowMode`) and viewer (`ViewerSubject.C`: the vis window’s toolbar actions are passed to the recorder); `scratch/ch32/verify_agent/t_zoom.py`, `t1_cli.py` (VisIt 3.5.0, no window).

FINDING YOUR WAY WITHOUT A MANUAL (1/2): FUNCTIONS AND PLUGINS

```
>>> Version()                    # '3.5.0'
>>> apropos("lineout")           # functions whose name or help mentions it:
['ClearReferenceLines', 'GetGlobalLineoutAttributes', 'GetPlotInformation', 'Lineout', '
  LineoutAttributes',
 'ResetLineoutColor', 'SetGlobalLineoutAttributes', 'SetWindowMode']
>>> help(Lineout)                # its synopsis, arguments, return value and an example
>>> PlotPlugins()                # ('Boundary', 'Contour', 'Curve', ... 'Vector', 'Volume'): 18
  plots
>>> OperatorPlugins()            # ('AMRStitchCell', 'AxisAlignedSlice4D', 'BoundaryOp', 'Box',
  ...)
```

`apropos` searches the names and the help texts; `help` prints a function's synopsis, arguments, return value and example. Next: the attributes objects.

FINDING YOUR WAY WITHOUT A MANUAL (2/2): ATTRIBUTES, METADATA, PLOTS

```
>>> p = PseudocolorAttributes(); print(p)
scaling = Linear  # Linear, Log, Skew
skewFactor = 1
limitsMode = OriginalData  # OriginalData, ActualData
minFlag = 0
...
>>> md = GetMetaData("viz_IB2d/dumps.visit"); print(md.numStates, md.times[-1])
>>> ListPlots()                  # the plots of the window, with their databases and variables
>>> len([f for f in dir(p) if f[0] != "_"])    # 165 names: a Get and a Set for each of the 55
  fields, and more
```

Printing an attributes object lists every field with its value and, for the choices, the names allowed (`p.Linear`, `p.Log`, ...). That is the recorder's format, and the quickest documentation there is.

`wsl/final/ch32_cli_helpers.sh`, `ch32_dir_sliders.sh` (VisIt 3.5.0).

THE INTERFACE IN ONE PAGE

task	functions (chapters where they, or their GUI counterparts, are used)
data	`OpenDatabase`, `ActivateDatabase`, `CloseDatabase`, `ReplaceDatabase`, `GetMetaData` (26–31, 33)
plots	`AddPlot`, `DeleteAllPlots`, `SetActivePlots`, `HideActivePlots`, `DrawPlots`, `ListPlots` (26–31, 33)
attributes	`PseudocolorAttributes()`, `MeshAttributes()`, `VectorAttributes()`, ..., `SetPlotOptions`; `BoxAttributes()`, ..., `SetOperatorOptions` (26, 29)
operators	`AddOperator("Box")`, `"Slice"`, `"Resample"`, `"Displace"`, `"IntegralCurve"` (27–29, 31, 33)
expressions	`DefineScalarExpression`, `DefineVectorExpression` (26–29)
time	`TimeSliderGetNStates`, `SetTimeSliderState`, `GetTimeSliders`, `SetActiveTimeSlider`, `CreateDatabaseCorrelation` (26, 30)
view	`GetView2D`/`SetView2D` (`windowCoords`), `GetView3D`/`SetView3D` (27, 30)
annotation	`AnnotationAttributes()`, `SetAnnotationAttributes`, `CreateAnnotationObject("Text2D")` (26, 30)
saving	`SaveWindowAttributes()`, `SetSaveWindowAttributes`, `SaveWindow`; `ExportDatabase` (30, 33)
numbers	`Query`, `GetQueryOutputValue`, `QueryOverTime`, `Lineout`, `NodePick`, `GetPlotInformation` (29)
sessions	`SaveSession`, `RestoreSession` (26, 30)

THE PATTERN, AND WHAT THIS SERIES DOES NOT CALL

```
p = PseudocolorAttributes()            # defaults; GetPlotOptions() would give the active plot's
p.colorTableName = CT; p.centering = p.Nodal
p.minFlag = 1; p.min = lo; p.maxFlag = 1; p.max = hi      # fixed limits
SetPlotOptions(p)                      # nothing changes on screen before this
```

Pattern: make an attributes object (from its constructor for defaults, or `Get...()` for the current state), change fields, hand it back with `Set...()`. Nothing changes on screen until you do. No script of this series calls `CloseDatabase`, `ReplaceDatabase`, `HideActivePlots`, `CreateDatabaseCorrelation` or `RestoreSession` (`ListPlots` only on slide 12): their GUI counterparts are in chapter 26.

`examples/analysis/visit_movie_frames.py`, lines 99–101; `GetPlotOptions()`: `wsl/final/ch32_getplotoptions.sh` (VisIt 3.5.0); the fact-check's search of chapters 26–33 and their scripts (`scratch/ch32/verify_agent/api_chapters.py`).

ERRORS: WHAT VISIT TELLS YOU, AND WHAT IT DOES NOT

```
r = OpenDatabase("/no/such/file.visit")    # prints "VisIt could not read from the file ...",
  returns 0
print(GetLastError())                      # the same message, as a string
r = OpenDatabase("viz_IB2d/dumps.visit")   # one that exists: returns 1
r = AddPlot("Pseudocolor", "NoSuchVariable")   # returns 0: "An invalid variable (NoSuchVariable)
  was specified."
r = DrawPlots()                            # returns 1 all the same
name = SaveWindow()                        # returns the file written: 'visit0000.png' (defaults:
  working folder)
```

- Most functions return 1 for success and 0 for failure; check the ones that matter (`OpenDatabase`, `AddPlot`) and stop the script, rather than draw nothing for an hour.
- `DrawPlots()` said 1 after a failed `AddPlot`: a return value is about that call only.
- Wrap the script in `try: ... finally: sys.exit()` so that an exception ends the run instead of leaving the CLI waiting.

`wsl/final/ch32_cli_helpers.sh` (VisIt 3.5.0); the listing run as printed: `scratch/ch32/verify_agent/t_errors2.py`.

WHAT VISIT DOES NOT REPORT, AND ITS LOGS

- What VisIt does not report as an error (chapters 29–30): a missing output folder (“Saved” and nothing written), a time query on a boxed plot 8 % low (the median; 25–31 % on frames of reversed flow), the Subset plot VisIt adds on opening drawing coloured lines, frames padded to 30 per second (at `-fps 10` a movie then shows 8.3 of your frames per second: 51 frames last 6.12 s). The CLI’s exit status reports nothing either: 250 whatever happened (slide 5).
- `-debug 1` to `-debug 5` write logs of each VisIt process to the working folder (`A.cli.1.vlog`, `A.viewer.1.vlog`, `A.mdserver.1.vlog`, `A.engine_ser.1.vlog`).

Chapters 29 and 30; `wsl/final/ch32_cli_helpers.sh`, `scratch/ch32/verify_agent/v9_exitcodes.sh` (VisIt 3.5.0).

TIME: THE LOOP EVERY SCRIPT NEEDS

```
md = GetMetaData(DB)                    # times, cycles, variables
N = md.numStates
for k in range(N):                      # from dump 0
    for ts in GetTimeSliders():         # every slider
        SetActiveTimeSlider(ts)
        SetTimeSliderState(k)
    t = md.times[k]
    # ... a query, a picture, a lineout at time t
```

- The 2018 loop (`TimeSliderNextState()` before the first save) skipped dump 0; this one does not (chapter 30).
- With two databases open VisIt has a slider for each and one for their correlation; setting only the active one leaves the others behind (chapter 30).
- `md.times` are the dumps' times (`0.0, 0.04, ..., 2.0` for IB/11): write them next to every number.
- `help(GetTimeSliders)` says it returns a tuple of strings; 3.5.0 returns a dictionary, slider name to state. A loop over it gets the names, as here.

The 2018 deck: "Add a loop to go through all times and save an image", with `k=1`, `N=2` and "#Note we advance in time to skip t=0".

"8 – Visit from command line and Python" (2018), slide 13; the loop of `examples/analysis/visit_movie_frames.py`; `GetTimeSliders()`: `wsl/final/ch32_dir_sliders.sh` (VisIt 3.5.0).

PYTHON 2 SCRIPTS IN VISIT 3

Python 2 (VisIt 2.x to 3.1)	Python 3 (VisIt 3.2 on)	why
`print "t =", t`	`print("t =", t)`	a function now
`n = N/2` with integers	`n = N // 2`	`/` always gives a float
`xrange(N)`	`range(N)`	`xrange` is gone
`"%s" % k`	`"%s" % k`, `f"{k}"`	both work; f-strings are new
`d.has_key(k)`	`k in d`	
`except Exception, e:`	`except Exception as e:`	syntax
tabs and spaces mixed in a block	one or the other	Python 3 refuses the mix (`TabError`)

- The 2018 and 2020 scripts that chapters 30–32 draw on contain nothing above that Python 3 refuses: they compile as Python 3 as they are, tab-indented loops included. Their VisIt functions all exist in 3.5.0 (24 checked), and so do their attribute fields (about 100), except three names of the 2020 Vector plot's: the field `lineStyle` and its value `SOLID` are gone, and `colorByMag` is now `colorByMagnitude`; setting `colorByMag` stops the script ("NameError: name 'colorByMag' is not defined"). `print` an object to see its current fields.
- VisIt's `-py2to3` option ("Enable limited on-the-fly Python 2 to 3 conversion support", `visit -help`) can run some old scripts unchanged; converting them is better.

VisIt 3.5.0 `visit -help`; `wsl/final/ch32_api_check.sh`, `ch32_vector_fields.sh`; the ports in `examples/analysis` (chapters 29–33).

NUMPY INSIDE VISIT'S CLI

```
import numpy as np                                  # in VisIt 3.5.0's CLI: Python 3.13.9, NumPy 2.3.4
DefineScalarExpression("U_x", "U[0]")
AddPlot("Pseudocolor", "U_x"); DrawPlots(); SetTimeSliderState(40)       # IB/11, t = 1.6 s
Lineout(start_point=(0, 0.10, 0), end_point=(0, 0.20, 0), use_sampling=1, num_samples=50)
SetActiveWindow(2)                                  # the lineout's curve window
xy = np.array(GetPlotInformation()["Curve"]).reshape(-1, 2)    # 51 (distance, U_x) pairs
q = np.trapezoid(xy[:, 1], xy[:, 0])                # 5.712e-3 m^2/s across the top vessel (chapter 29's averages.csv, t = 1.6)
```

- The 2018 deck said VisIt's Python was "a basic Python 2.x environment . . . NumPy and matplotlib in particular are absent". VisIt 3.5.0's CLI imports `numpy` (2.3.4), `vtk` (9.5.0), `PIL` (Pillow 12.0.0) and `visit_utils`; `matplotlib`, `scipy`, `h5py`, `netCDF4`, `pyvista` and `pandas` are absent.
- Without `use_sampling=1` the lineout samples where the line meets the mesh (52 points here, 5.709e-3).
- So arithmetic on VisIt's numbers can stay in the script (NumPy's `polyfit` and `linalg.lstsq` too); plots and SciPy's fits go outside: write CSV or `.curve` files and read them in your own Python (chapter 29's `read_curve.py`), or use the next slide's route.
- Do not `pip install` into VisIt's Python; keep VisIt's install as it came.

`wsl/final/ch32_syspython2.sh`, `ch32_numpy_lineout.sh`, `ch32_api_check.sh`, `visit_py_pkgs.sh` (VisIt 3.5.0, Linux); the calls as in `examples/analysis/visit_queries.py`.

VISIT INSIDE YOUR OWN PYTHON

```
export PATH=/opt/visit/bin:$PATH                                    # `visit` must be found
export LD_LIBRARY_PATH=/opt/visit/3.5.0/linux-x86_64/lib            # its libraries too
python3 my_analysis.py
```

```
import sys
import matplotlib.pyplot as plt              # your Python's packages, in the same script: import
  them before visit
sys.path.append("/opt/visit/3.5.0/linux-x86_64/lib/site-packages")
import visit
visit.LaunchNowin()                          # or visit.AddArgument("-nowin"); visit.Launch()
visit.OpenDatabase("viz_IB2d/dumps.visit")
visit.AddPlot("Pseudocolor", "Omega"); visit.DrawPlots(); visit.SetTimeSliderState(50)
visit.Query("MinMax"); print(visit.GetQueryOutputValue())   # (-396.98..., 412.50...), as in the
  CLI
```

- The functions are the CLI's, prefixed by `visit.`. The 2018 Windows hotfix to `frontend.py` was not tried.
- Why the two variables, and what happened when this ran: next slide.

"8 – Visit from command line and Python" (2018), slides 15–18; `wsl/final/ch32_syspython*.sh` (paths shortened to `/opt/visit`).

VISIT INSIDE YOUR OWN PYTHON: WHAT WAS MEASURED

- C. Strickland's route of 2018, measured with Ubuntu's Python 3.12: without `LD_LIBRARY_PATH`, `import visit` stops with "ImportError: libviewerproxy.so: cannot open shared object file"; without VisIt's `bin` on the `PATH`, `LaunchNowin()` with "Unable to launch program visit". With both it ran and gave the same MinMax every time, in a few runs after a harmless `BrokenPipeError` traceback from VisIt's launcher.
- Python then often aborted on exit, after the script's last line ("Aborted (core dumped)", status 134): in 12 of 18 runs with matplotlib imported after `visit`, 1 of 9 with it imported first (as on the previous slide), none of 10 with neither matplotlib nor NumPy. The output was complete every time; the exit status cannot be trusted.
- `export LD_LIBRARY_PATH=...` holds for every program started from that shell afterwards, and system programs then load VisIt's own copies of their libraries. `LD_LIBRARY_PATH=... python3 my_analysis.py` sets it for that one command (whether that changes the abort on exit was not tested).

`wsl/final/ch32_syspython*.sh`; `scratch/ch32/verify_agent/v5_runs.sh`, `v6_import.sh`, `v7_import_repeat.sh`, `v7b_import_repeat.sh` (Ubuntu 24.04's `python3` 3.12.3, VisIt 3.5.0).

MANY RUNS: ONE SCRIPT, A LOOP OUTSIDE

A parameter sweep (chapter 3; IB/01's job array) leaves one folder per run (`run_1`, `run_2`, ...). Run the same VisIt script on each:

- A shell loop: `for d in run_*; do visit -nowin -cli -s visit_queries.py -- $d/viz_IB2d/dumps.visit $d/analysis; done`. Each run gets a fresh VisIt; a failure stops one run, not the sweep. Give `box=`, `point=` and `line=` for your geometry: the defaults are IB/11's top vessel, and on another run they measure nothing meaningful, without an error.
- A Slurm array: one array task per run folder, the same script (chapter 31's batch job, with `$SLURM_ARRAY_TASK_ID` choosing the folder as in IB/01's `job_array.slurm`, chapter 3).
- One VisIt for all: `CloseDatabase` and `OpenDatabase` (or `ReplaceDatabase`, which keeps the plots) inside one script. Fewer start-ups (VisIt takes a few seconds to start), but one error ends everything.
- Write each run's numbers to its own folder (`averages.csv`, `frames.csv`), then gather them in Python or MATLAB: the record of chapter 29 (protocol, data sheet, report).

> Test on one run, then on two, then on all; and count the outputs (frames, rows) of every run, not only of the first.

THE SCRIPTS OF THIS SERIES, AS EXAMPLES

examples/analysis/	what it shows	chapter
visit_movie_frames.py	arguments, the structure's meshes found from the metadata, fixed limits from a query, a Text2D time stamp, a loop over time, a CSV of what was written, first= to number a later chunk on (chapter 31)	30
visit_queries.py	a Box, queries frame by frame and through time, a node pick through time, lineouts, curves saved as .curve files and their values read from the plots, an expression for the normal velocity	29
visit_3d.py	3D views by numbers, slices, isosurfaces, vectors on a slice, expressions	27
resample_export.py	the Resample operator and ExportDatabase in a loop, and a fix for an export quirk	33
visit_frames.slurm	the CLI in a batch job	31
make_movie.py, read_curve.py	plain Python on VisIt's outputs: ffmpeg, .curve files	30, 29

Each Python script starts with a docstring that says how to run it and what it writes (the Slurm script, with comments). The VisIt ones print what they are doing (flushed, so a batch log shows it) and end with sys.exit(), inside try/finally in visit_movie_frames.py and visit_queries.py. All were run on this series' data before they were published (visit_frames.slurm with bash on this computer, not yet on Puma: chapter 31).

WHEN A SCRIPT GOES WRONG

what you see	what to do
the CLI waits at >>> after the script (started from a terminal; a batch job's input is `/dev/null`)	end it with `sys.exit()`, inside `finally:`
`SyntaxError` on `print "..."`	a Python 2 script: slide 18
`TabError` or `IndentationError`	tabs mixed with spaces (recorded code pasted into an editor)
`ValueError` reading `key=value` arguments	`-np` added `-engineargs`: slide 6
nothing drawn, and the script ran on	a failed `AddPlot` earlier (its message is in the log): check its return value, `GetLastError()`
"An invalid variable"	the variable's name in this database: `GetMetaData`; IBAMR 0.19 writes `U`, not `U_x` (chapter 26)
the recorded path does not open	another machine's path (the `localhost:` before it can stay; slide 9)
a picture of the whole domain	the script never sets the view: keep the recorded `SetView2D` (slide 10), or set `windowCoords`
"VisIt's viewer has terminated abnormally!"	a recorded `SetWindowMode("zoom")`: delete it (slide 10)
`ImportError: libviewerproxy.so` in your Python	`LD_LIBRARY_PATH` (slide 20)
`ModuleNotFoundError` for `matplotlib` in the CLI	not in VisIt's Python: write the numbers out, or use your own Python

EXERCISES

1. Record a picture of IB/11's vorticity in the GUI (**Record**, **Stop**); cut the recording down to the lines that matter; run it with `visit -nowin -cli -s` and compare the pictures.
2. Add a loop over time that writes one picture per dump, numbered without gaps, and a CSV of the times.
3. Use `apropos` and `help` to find how to save a curve window as a `.curve` file; do it.
4. Port one of the 2018 or 2020 scripts quoted in chapters 31–32 to VisIt 3.5 (paths, dump 0, the renamed fields of slide 18; their syntax is already Python 3).
5. Compute the flow rate across a line with NumPy inside the CLI, and check it against chapter 29's number (its `averages.csv`: 5.712×10^{-3} m^2/s at $t = 1.6$ s).
6. Run the same analysis from your own Python with `import visit`, and plot the result with matplotlib in the same script.
7. Run `visit_queries.py` on every run of a small sweep with a shell loop; gather the results in one table.

SUMMARY

- `visit -nowin -cli -s script.py -- arguments` is the way to repeat anything; the GUI's **Record** writes the first draft.
- VisIt 3.5's CLI is Python 3.13 with NumPy; old scripts may need Python 3 syntax (the decks' own do not), need new paths, and should save dump 0.
- `help`, `apropos`, `print(attributes)`, `GetMetaData`, `ListPlots`: the documentation inside VisIt.
- Check return values and `GetLastError()`; some failures are silent, so count what a script writes.
- Your own Python can drive VisIt with `import visit`, given VisIt's `bin` on the `PATH` and its `lib` on `LD_LIBRARY_PATH`.

> A script is the record of how a picture or a number was made. Keep it with the run.

REFERENCES AND CREDITS

- L. A. Miller, “8 – Visit from command line and Python” (2018; with C. Strickland’s notes on controlling VisIt from Python), “6 – Resampling AMR grids to Uniform Grid, exporting to .vtk, and recording in Python” (2018), “3B – Analyzing your IBFE simulation” (2018).
- G. Hobson, “Post-Processing IBAMR Simulation Data in VisIt on a Computing Cluster” (2020), slides 6–7 and 13.
- VisIt 3.5.0, https://visit-dav.github.io/visit-website/; its user manual’s chapter on the Python interface, https://visit-sphinx-github-user-manual.readthedocs.io/; H. Childs et al., “VisIt: An end-user tool for visualizing and analyzing very large data”, in *High Performance Visualization*, CRC Press (2012) 357–372.
- The runs: IB/11 (`TARGET_VP`), IBAMR 0.19.0; the measurements `wsl/final/ch32_*.sh` in the working folder.